

1 Basic Definitions

Learning: Find a model f that maps from inputs to outputs given a labeled training dataset.

Inference: Use a model f to predict output y given new example $\tilde{x}$

Model: Function f mapping input $\tilde{x}$ to output prediction y .

Model family: Set of models that share the same structure but differ in parameters (e.g. layers in a NN)

Accuracy: $\frac{\sum_{i=1}^N \mathbb{I}[t^{(i)}=y^{(i)}]}{N}$

Parameter: Internal variable of model learned from data whilst training

Hyperparameter: Settings chosen before training that controls how the learning process works.

Parametric model: Any model with the parameter counts NEVER being $\Omega(N)$ (invariant to the data count)

Precision: $\text{TPR} = \frac{\text{TP}}{\text{TP} + \text{FN}} = p$ (actually + |pred+)

Recall: $\frac{\text{TP}}{\text{TP} + \text{FN}} = p$ (pred + |actually +)

1.1 KNN

Find $t^{(c)}$ corresponding to

$\tilde{x}^{(c)} = \arg \min_{\tilde{x}^{(i)} \in \text{train set}} \text{distance}(\tilde{x}^{(i)}, \tilde{x})$

Adv: Simple, no training, works for reg. and class, adapts easily to new data.

Disadvantage: Bad w/ high dims, sensitive to scaling, slow for large datasets, must store logs proportional to data. Make k odd or chose a tiebreaker.

2 Decision Trees

Large trees overfit, small trees underfit, trace the tree on inference. $H(X) = -\sum_{x \in X} p(x) \log_2 p(x)$,

$H(X|Y=y) = -\sum_{x \in X} p(x|Y=y) \log_2 p(x|Y=y)$.

Expected conditional entropy:

$H(X|Y) = -\sum_{y \in Y} p(y) \sum_{x \in X} p(x|y) \log_2 p(x|y)$,

$IG(X|Y) = H(X) - H(X|Y)$

Properties: Entropy nonnegative, $H(Y|X) \leq H(Y)$, Y, X independent $\Rightarrow IG(Y|X) = 0$, fully determinant rids entropy.

Splitting: Keep splitting until all examples have same label, when no features left, when depth limit is reached (hyperparam), when loss stops decreasing enough (hyperparam)

3 Linear Regression

Loss measures how off **one feature** is, cost measures

all. $\mathcal{L}(y, t) = \frac{1}{2}(y - t)^2$, $\mathcal{E}(\tilde{w}) = \frac{1}{N} \sum_{i=1}^N \mathcal{L}(y^{(i)}, t^{(i)})$.

Hence, for linear model $\tilde{y} = W\tilde{x}$.

• $\mathcal{E}(\tilde{w}) = \frac{1}{2N} \sum_{i=1}^N (\tilde{w}^T \tilde{x} - t^{(i)})^2$

• $\mathcal{E}(\tilde{w}) = \frac{1}{2N} (\tilde{w} - \tilde{t})^T (\tilde{w} - \tilde{t})$

Gradients:

• $\frac{\partial \mathcal{E}}{\partial w_j} = \frac{1}{N} \sum_{i=1}^N (\tilde{w}^T \tilde{x}^{(i)} - t^{(i)}) x_j^{(i)}$

• $\nabla_{\tilde{w}} \mathcal{E}(\tilde{w}) = \frac{1}{N} \sum_{i=1}^N (\tilde{w}^T \tilde{x}^{(i)} - t^{(i)}) \tilde{x}^{(i)}$

• $\nabla_{\tilde{w}} \mathcal{E}(\tilde{w}) = \frac{1}{N} X^T (\tilde{w} - \tilde{t})$

W/ bias: $\frac{\partial \mathcal{E}(\tilde{w})}{\partial w_j} = \frac{1}{N} \sum_{i=1}^N (\tilde{w}^T \tilde{x}^{(i)} + b - t^{(i)}) x_j^{(i)}$,

$\frac{\partial \mathcal{E}(\tilde{w})}{\partial b} = \frac{1}{N} (\tilde{w} - \tilde{t})$

Regularizer: $\frac{\lambda}{2} \sum_{j=0}^D w_j^2$, $\nabla_{\tilde{w}} \left(\frac{\lambda}{2} \tilde{w}^T \tilde{w} \right) = \lambda \tilde{w}$

Stochastic GD: Randomly pick a training example, very noisy, $\tilde{w} \leftarrow \tilde{w} - \alpha \frac{\partial \mathcal{L}}{\partial \tilde{w}}$. Mini-batch GD falls in-between. Batch size is a hyperparameter

4 Logistic Regression

• $z = \mathbf{w}^T \mathbf{x}$, $y = \mathbb{I}[z \geq 0]$

Loss functions:

• 0-1 $\mathcal{L}_{0-1}(y, t) = \mathbb{I}[y \neq t]$. Hard to optimize

• $\mathcal{L}_{SE}(z, t) = \frac{1}{2}(z - t)^2$. Large loss for v. correct

• $y = \sigma(z) = \frac{1}{1+e^{-z}}$, $\mathcal{L}_{SE}(y, t) = \frac{1}{2}(y - t)^2$, prediction always $[0, 1]$, ∇ small at v. large magnitudes

• Cross entropy:

$$\mathcal{L}_{CE}(y, t) = \begin{cases} -t \log(y) & t = 1 \\ -(1-t) \log(1-y) & t = 0 \end{cases}$$

CE loss derivation, $z = \tilde{w}^T \tilde{x}$. $y = \sigma(z) = \frac{1}{1+e^{-z}}$.

$\sigma'(z) = \sigma(z)(1 - \sigma(z))$. $\frac{\partial y}{\partial z} = y(1 - y)$.

$\mathcal{L}_{CE}(y, t) = -t \log(y) - (1-t) \log(1-y)$.

GD update: $\mathcal{E}(\tilde{w}) = \frac{1}{N} \sum_{i=1}^N \mathcal{L}_{CE}(y^{(i)}, t^{(i)})$.

• $\frac{\partial \mathcal{L}_{CE}}{\partial w_j} = \frac{\partial \mathcal{L}_{CE}}{\partial y} \frac{\partial y}{\partial z} \frac{\partial z}{\partial w_j}$

• $= \left(-\frac{t}{y} + \frac{1-t}{1-y} \right) y(1-y) x_j = (y-t) x_j$

• Vectorized: $\tilde{w} \leftarrow \tilde{w} - \frac{\alpha}{N} X^T (\tilde{w} - \tilde{t})$

4.1 Softmax Gradient

The softmax model is

• $\mathbf{z} = W\mathbf{x}$ $\mathbf{y} = \text{softmax}(\mathbf{z})$

• $y_k = \frac{\exp(z_k)}{\sum_{m=1}^K \exp(z_m)}$

• $\mathcal{L}(\mathbf{y}, \mathbf{t}) = -\mathbf{t}^T \log(\mathbf{y})$

Derive: $\frac{\partial \mathcal{L}}{\partial \mathbf{w}_m}$. Derivation:

• $L = -\sum_{k=1}^K t_k \log(y_k)$

• $\frac{\partial L}{\partial y_k} = -\frac{t_k}{y_k}$

• $k \neq m \Rightarrow \frac{\partial y_k}{\partial z_m} = \frac{\partial}{\partial z_m} \left(\frac{\exp(z_k)}{\sum_{n=1}^K \exp(z_n)} \right)$

• $= \frac{-\exp(z_k) \exp(z_m)}{(\sum_{n=1}^K \exp(z_n))^2} = -y_k y_m$

• $k = m \Rightarrow \frac{\partial y_k}{\partial z_k} = \frac{\partial}{\partial z_k} \left(\frac{\exp(z_k)}{\sum_{n=1}^K \exp(z_n)} \right)$

• $= \frac{\exp(z_k) ((\sum_{n=1}^K \exp(z_n)) - \exp(z_k))}{(\sum_{n=1}^K \exp(z_n))^2}$

• $= y_k (1 - y_k)$

• $\frac{\partial y_k}{\partial z_m} = \begin{cases} -y_k y_m & \text{if } k \neq m \\ y_k (1 - y_k) & \text{if } k = m \end{cases}$

• $\frac{\partial L}{\partial z_m} = \sum_{k=1}^K \frac{\partial L}{\partial y_k} \frac{\partial y_k}{\partial z_m}$

• $= \sum_{k=1}^K \begin{cases} t_k y_m & \text{if } k \neq m \\ t_k (y_k - 1) & \text{if } k = m \end{cases}$

• $= t_1 y_m + \dots + t_m (y_m - 1) + \dots + t_K y_m$

Since $\mathbf{t}$ is one-hot: if $t_m = 1$, term left in sum is

$t_m (y_m - 1) = y_m - 1 = y_m - t_m$. If $t_k = 1$ where $k \neq m$, then $t_m = 0 \Rightarrow t_k y_m = y_m = y_m - t_m$. Either way, we

have that $\frac{\partial L}{\partial z_m} = y_m - t_m$. Hence:

$\frac{\partial L}{\partial \mathbf{w}_m} = \frac{\partial L}{\partial z_m} \frac{\partial z_m}{\partial \mathbf{w}_m} = (y_m - t_m) \mathbf{x}$

4.2 Softmax K=2 Proof

$K = 2$, $y_k = \frac{e^{z_k}}{\sum_{m=1}^K e^{z_m}}$ correspond to $y = \frac{1}{1+e^{-z}}$.

• $y_1 = \frac{e^{z_1}}{e^{z_1} + e^{z_2}} = \frac{e^{z_1}}{e^{z_1}(1+e^{z_2-z_1})} = \frac{1}{1+e^{-(z_1-z_2)}}$

5 NNs

An MLP is a feed-forward network composed of an input layer, one or more hidden layers, and output layers.

AND: $x_1 + x_2 - 1.5$, OR: $x_1 + x_2 - 0.5$

XOR: $(x_1 \wedge \neg x_2) \vee (\neg x_1 \wedge x_2)$, alt:

$(x_1 \vee x_2) \wedge (\neg(x_1 \wedge x_2))$

Brute forcing:

$[-N_{\text{TRUE}} + 0.5, -999, -999, 1, 1, -999]$

6 Backprop

The goal is to find good weights for an NN by minimizing a loss function. Computing gradients traditionally is challenging because a network can have millions of weights. Using chain rule results in massive inefficiency due to repeated calculations of the same intermediate quantities. Backpropagation efficiently computes all the derivatives in a single pass by reusing intermediate results.

Forward pass. System computes and stores pre-activations $\mathbf{z}$ and activations $\mathbf{a}$ for every layer, proceeding from the input to the output. Needed b/c these values must be used later

Backward pass. Algorithm starts by computing the gradient of the loss function with respect to the output layer's pre-activations. It recursively computes gradients for hidden layers, moving backwards from the output to the input. It uses the multivariate chain rule to propagate error terms through the network.

6.1 Backprop Vectorizer

Write the 1 vector in blackboard bold!

• $z_j^{(1)} = \sum_{i=1}^{D^{(0)}} W_{ji}^{(1)} a_i^{(0)} + b_j^{(1)}$

- $\mathbf{z}^{(1)} = W^{(1)} \mathbf{a}^{(0)} + \mathbf{b}^{(1)}$

- $\mathbf{Z}^{(1)} = A^{(0)} (W^{(1)})^T + \mathbf{1}_N (\mathbf{b}^{(1)})^T$

• $a_j^{(1)} = \text{ReLU}(z_j^{(1)})$

- $\mathbf{a}^{(1)} = \text{ReLU}(\mathbf{z}^{(1)})$, $A^{(1)} = \text{ReLU}(\mathbf{Z}^{(1)})$

• $a_k^{(2)} = \frac{e^{z_k^{(2)}}}{\sum_{k'=1}^{D^{(2)}} e^{z_{k'}^{(2)}}}$

- $\mathbf{a}^{(2)} = \frac{\exp(\mathbf{z}^{(2)})}{\mathbf{1}_{D^{(2)}}^T \exp(\mathbf{z}^{(2)})}$

- $A^{(2)} = \frac{\exp(\mathbf{Z}^{(2)})}{\exp(\mathbf{Z}^{(2)}) \mathbf{1}_{D^{(2)}}}$

• $\mathcal{L}(\mathbf{a}^{(2)}, \mathbf{t}) = -\sum_{k=1}^{D^{(2)}} t_k \log(a_k^{(2)})$

- $\mathcal{L}(\mathbf{a}^{(2)}, \mathbf{t}) = -\mathbf{t}^T \log(\mathbf{a}^{(2)})$

- $\mathcal{L} = -\frac{1}{N} \mathbf{1}_N^T (\mathbf{T} \odot \log A^{(2)}) \mathbf{1}_{D^{(2)}}$

• $\frac{\partial \mathcal{L}}{\partial z_k^{(2)}} = a_k^{(2)} - t_k$

- $\nabla_{\mathbf{z}^{(2)}} \mathcal{L} = \mathbf{a}^{(2)} - \mathbf{t}$

- $\nabla_{\mathbf{Z}^{(2)}} \mathcal{L} = \frac{1}{N} (A^{(2)} - \mathbf{T})$

• $\frac{\partial \mathcal{L}}{\partial w_{kj}^{(2)}} = \frac{\partial \mathcal{L}}{\partial z_k^{(2)}} a_j^{(1)}$

- $\nabla_{W^{(2)}} \mathcal{L} = (\nabla_{\mathbf{z}^{(2)}} \mathcal{L}) (\mathbf{a}^{(1)})^T$

- $\nabla_{W^{(2)}} \mathcal{L} = (\nabla_{\mathbf{Z}^{(2)}} \mathcal{L})^T A^{(1)}$

• $\frac{\partial \mathcal{L}}{\partial a_j^{(1)}} = \sum_{k=1}^{D^{(2)}} \frac{\partial \mathcal{L}}{\partial z_k^{(2)}} W_{jk}^{(2)}$

- $\nabla_{\mathbf{a}^{(1)}} \mathcal{L} = (W^{(2)})^T (\nabla_{\mathbf{z}^{(2)}} \mathcal{L})$

- $\nabla_{A^{(1)}} \mathcal{L} = (\nabla_{\mathbf{Z}^{(2)}} \mathcal{L})^T W^{(2)}$

• $\frac{\partial \mathcal{L}}{\partial z_j^{(1)}} = \frac{\partial \mathcal{L}}{\partial a_j^{(1)}} \text{ReLU}'(z_j^{(1)})$

- $\nabla_{\mathbf{z}^{(1)}} \mathcal{L} = (\nabla_{\mathbf{a}^{(1)}} \mathcal{L}) \odot \text{ReLU}'(\mathbf{z}^{(1)})$

- $\nabla_{\mathbf{Z}^{(1)}} \mathcal{L} = (\nabla_{A^{(1)}} \mathcal{L}) \odot \text{ReLU}'(\mathbf{Z}^{(1)})$

• $\frac{\partial \mathcal{L}}{\partial z_j^{(1)}} = \frac{\partial \mathcal{L}}{\partial a_j^{(1)}} \text{ReLU}'(z_j^{(1)})$

- $\nabla_{\mathbf{z}^{(1)}} \mathcal{L} = (\nabla_{\mathbf{a}^{(1)}} \mathcal{L}) \odot \text{ReLU}'(\mathbf{z}^{(1)})$

- $\nabla_{\mathbf{Z}^{(1)}} \mathcal{L} = (\nabla_{A^{(1)}} \mathcal{L}) \odot \text{ReLU}'(\mathbf{Z}^{(1)})$

6.2 Vectorization Utilities

Pay attention to the dimensionality.

• $\sum_{i=1}^P \mathbf{m}_i \mathbf{u}_i = \mathbf{M} \mathbf{u}$

• $\alpha_i = \mathbf{m}_i^T \mathbf{u} \Rightarrow \alpha = \mathbf{M} \mathbf{u}$

• $\sum_{i=1}^P \mathbf{u}_i \mathbf{v}_i \mathbf{m}_i = \mathbf{M}^T (\mathbf{u} \odot \mathbf{v})$

• $\sum_{i=1}^P \sum_{j=1}^Q \mathbf{u}_i \mathbf{v}_j \mathbf{m}_{ij} = \mathbf{u}^T \mathbf{M} \mathbf{v}$

• $\sum_{i=1}^Q \sum_{k=1}^Q \mathbf{u}_i \mathbf{m}_{ik} \mathbf{u}_k = \mathbf{u}^T \mathbf{M} \mathbf{u}$

• $\sum_{i=1}^P \sum_{j=1}^Q x_i y_j z_{ij} = \mathbf{x}^T \mathbf{Z} \mathbf{y}$

• $\sum_{i=1}^P y_i z_i x_i = \mathbf{Z}^T (\mathbf{x} \odot \mathbf{y})$

Sum $\rightarrow \leftarrow$ is $X \mathbf{1}_P$. Sum $\downarrow$ is $\mathbf{1}_N^T X$. $\mathbf{u} \odot \mathbf{v} = \text{diag}(\mathbf{u}) \mathbf{v}$.

Row-wise mult: $\text{diag}(\mathbf{u}) \mathbf{X}$, col: $\mathbf{X} \cdot \text{diag}(\mathbf{v})$;

$\begin{bmatrix} -\mathbf{z}^T W - \\ - \\ - \end{bmatrix} = \mathbf{Z} W$

7 BV Decomposition

• Bias: model's inability to capture true relationship.

• Variance: Model's sensitivity to random fluctuations. Resample from training set and model changes by a lot

• Bayes error: Inherent noise in the problem

First: $\mathbb{E}_{\mathbf{x}, t, D} [(f(\mathbf{x}) - t)^2]$

$$\begin{aligned}
&= \mathbb{E} \left[\left(f(\mathbf{x}) - \bar{f}(\mathbf{x}) + \bar{f}(\mathbf{x}) - t \right)^2 \right] \\
&= \mathbb{E} \left[\left(f(\mathbf{x}) - \bar{f}(\mathbf{x}) \right)^2 \right] + 0 + \\
&= \mathbb{E} \left[\left(\bar{f}(\mathbf{x}) - t \right)^2 \right] \\
\text{Then: } \mathbb{E}_{\mathbf{x}, t, \mathcal{D}} \left[\left(\bar{f}(\mathbf{x}) - t \right)^2 \right] \\
&= \mathbb{E} \left[\left(\bar{f}(\mathbf{x}) - \bar{y}(\mathbf{x}) + \bar{y}(\mathbf{x}) - t \right)^2 \right] \\
&= \mathbb{E} \left[\left(\bar{f}(\mathbf{x}) - \bar{y}(\mathbf{x}) \right)^2 \right] + \\
&= \mathbb{E} \left[\left(\bar{y}(\mathbf{x}) - t \right)^2 \right]
\end{aligned}$$

7.1 Bagging

For random forests: using randomization aims to reduce correlation among trees so that their prediction errors are less likely to align. As a result, the ensemble achieves a greater reduction in variance and better generalization.

$$\begin{aligned}
V[y] &= V \left[\frac{1}{m} \sum_{i=1}^m y_i \right] \\
&= \frac{1}{m^2} V \left[\sum_{i=1}^m y_i \right] \\
&= \frac{1}{m^2} \sum_{i=1}^m V[y_i] = \frac{1}{m} V[y_i]
\end{aligned}$$

8 MLE

The denominator is the same for all classes and scales each numerator by the same amount.

$$p(x_j^{(i)} | c^{(i)}) = \left(p(x_j^{(i)} | c^{(i)} = 1) \right)^{c^{(i)}} \times \left(p(x_j^{(i)} | c^{(i)} = 0) \right)^{1-c^{(i)}}$$

$$\begin{aligned}
\theta_{j1} &= \frac{\sum_{i=1}^N c^{(i)} x_j^{(i)}}{\sum_{i=1}^N c^{(i)}} \\
\theta_{j0} &= \frac{\sum_{i=1}^N (1-c^{(i)}) x_j^{(i)}}{\sum_{i=1}^N (1-c^{(i)})} \\
\text{MAP: Multiply by every possible parameter.} \\
&\text{Preferable as it prevents overfitting to limited data,} \\
&\text{allows us to incorporate any prior belief about the} \\
&\text{parameter.}
\end{aligned}$$

9 GDA

$$\mathcal{N}(\mathbf{x}; \mu, \Sigma) = \frac{1}{(2\pi)^{\frac{D}{2}} |\Sigma|^{\frac{1}{2}}} \exp \left(-\frac{1}{2} \tilde{\mathbf{x}}^T \Sigma^{-1} \tilde{\mathbf{x}} \right)$$

10 K-Means

Update assignments, update centers. D features, N data points, K clusters, $\tilde{m}_k \in \mathbb{R}^D$ denotes the center of cluster k , $r_k^{(i)} \in \{0, 1\}$ denotes i is assigned to cluster k w/ center $\tilde{m}_k$. Hence, $\tilde{\tau}^{(i)}$ is one-hot. Global objective: $\mathcal{J}(\{\tilde{m}_k\}, \{\tilde{\tau}^{(i)}\}) =$

$$\min_{\{\tilde{m}_k\}, \{\tilde{\tau}^{(i)}\}} \sum_{i=1}^N \sum_{k=1}^K r_k^{(i)} \left\| \tilde{m}_k - \tilde{x}^{(i)} \right\|^2$$

Assignment step: for each point $\tilde{x}^{(i)}$, find $\tilde{\tau}^{(i)}$ (just brute-force):

$$\min_{\tilde{\tau}^{(i)}} \sum_{k=1}^K r_k^{(i)} \left\| \tilde{m}_k - \tilde{x}^{(i)} \right\|^2$$

Center step (change centers): for center $\tilde{m}_k$:

$$\arg \min_{\tilde{m}_k} \sum_{i=1}^N r_k^{(i)} \left\| \tilde{m}_k - \tilde{x}^{(i)} \right\|^2$$

$$\tilde{m}_k = \frac{\sum_{i=1}^N r_k^{(i)} \tilde{x}^{(i)}}{\sum_{i=1}^N r_k^{(i)}}$$

We stop K-means when the assignments no longer change $\rightarrow$ we have converged to a local minimum.

Local minimum convergence is guaranteed: each step never increases the objective, there's a finite no. of cluster assignments, algorithm must stop after finite no. of iterations. **If local minima is bad, restart with random cluster centers.**

11 EM

Soft K-means, clusters don't need to be a circle (allows elliptical clusters, and clusters are of a flexible size). A GMM represents $p(\tilde{\mathbf{x}})$ a mixture (weighted combination of multiple gaussian distributions) of K gaussian components:

$$p(\tilde{\mathbf{x}}) = \sum_{k=1}^K \pi_k \mathcal{N}(\tilde{\mathbf{x}} | \mu_k, \Sigma_k)$$

Where $\sum_{k=1}^K \pi_k = 1$, $\pi_k \geq 0$, $\forall k$

Generating data: pick $z^{(i)} \sim \text{Categorical}(\tilde{\pi})$, determine features $\tilde{x}^{(i)}; \tilde{x}^{(i)} | z^{(i)} \sim \mathcal{N}(\tilde{x} | \tilde{\mu}_{z^{(i)}}, \tilde{\Sigma}_{z^{(i)}})$.

Hence, generative model is

$$p(\tilde{\mathbf{x}}) = \sum_{k=1}^K \pi_k \mathcal{N}(\tilde{\mathbf{x}} | \tilde{\mu}_k, \tilde{\Sigma}_k). z^{(i)} \text{ is a latent variable and indicates which Gaussian component generates } \tilde{x}^{(i)}.$$

EM Algorithm, D features, N data points, K clusters, params: $\tilde{\theta} = \{\pi_k, \tilde{\mu}_k, \tilde{\Sigma}_k, k = 1 \dots K\}$. Each cluster k has a mixing weight, mean, and covariance.

$r_k^{(i)} \in [0, 1]$ is the responsibility that components k takes for explaining point i (latent variable $z^{(i)}$ indicates that generated point i)

Max this likelihood:

$$\log p(\mathcal{D} | \tilde{\theta}) = \sum_{i=1}^N \log \left(\sum_{k=1}^K \pi_k \mathcal{N}(\tilde{x}^{(i)} | \tilde{\mu}_k, \tilde{\Sigma}_k) \right)$$

GMMs are universal density approximators. With MLE, training likelihood can grow unbounded, hence **there could be a tiny-variance gaussian on one training example**. This is overfitting. Hence, this is a two-step descent

Expectation step: Compute the responsibility each cluster k takes for explaining point i

$$\begin{aligned}
r_k^{(i)} &= p(z^{(i)} = k | \tilde{x}^{(i)}, \tilde{\theta}) = \frac{p(z^{(i)} = k) p(\tilde{x}^{(i)} | z^{(i)} = k)}{p(\tilde{x}^{(i)})} \\
&= \frac{\pi_k \mathcal{N}(\tilde{x} | \tilde{\mu}_k, \tilde{\Sigma}_k)}{\sum_{k=1}^K \pi_k \mathcal{N}(\tilde{x} | \tilde{\mu}_k, \tilde{\Sigma}_k)}
\end{aligned}$$

Maximization step: Maximizes the probability that each cluster generates the data points assigned to it

$$\begin{aligned}
\pi_k &= \frac{1}{N} \sum_{i=1}^N r_k^{(i)}, \tilde{\mu}_k = \frac{\sum_{i=1}^N r_k^{(i)} \tilde{x}^{(i)}}{\sum_{i=1}^N r_k^{(i)}} \\
\tilde{\Sigma}_k &= \frac{\sum_{i=1}^N r_k^{(i)} (\tilde{x}^{(i)} - \tilde{\mu}_k) (\tilde{x}^{(i)} - \tilde{\mu}_k)^T}{\sum_{i=1}^N r_k^{(i)}}
\end{aligned}$$

12 PCA

Where $\tilde{\mathbf{x}} = \mathbf{U} \mathbf{z} = \mathbf{U} \mathbf{U}^T \mathbf{x}$

• Projecting & reconstructing a 2D vector $\mathbf{x} \in \mathbb{R}^2$ onto $\mathbf{u} \in \mathbb{R}^2$: $\tilde{\mathbf{x}} = (\mathbf{x}^T \mathbf{u}) \cdot \frac{\mathbf{u}}{\text{mag}}$

- The **projection** step is just $\mathbf{z} = \tilde{\mathbf{x}}^T \mathbf{u}$
- The **reconstruction** step is $\tilde{\mathbf{x}} = \mathbf{u} \mathbf{z}$
- The **representation** of $\mathbf{x}$ in basis $\mathbf{U}$ is $\mathbf{z} = \mathbf{U}^T \mathbf{x}$
- Reconstruction error $\|\mathbf{x} - \tilde{\mathbf{x}}\|^2$
- Before PCA, data is best centered since variance is about how data varies around mean, not origin.

$$\tilde{\mathbf{x}}^{(i)} = \mathbf{x}^{(i)} - \mu$$

• Optimal basis (both are equivalent):

- Max recons. Var.: $\max \frac{1}{N} \sum_{i=1}^N \|\tilde{\mathbf{x}}^{(i)}\|^2$
- Min recons. Err.: $\min \frac{1}{N} \sum_{i=1}^N \|\mathbf{x}^{(i)} - \tilde{\mathbf{x}}^{(i)}\|^2$

Why are the two conditions above equivalent?

- Prove that $\mathbf{x}^{(i)} - \tilde{\mathbf{x}}^{(i)}$ and $\tilde{\mathbf{x}}^{(i)}$ are orthogonal.
 - Since $\tilde{\mathbf{x}}^{(i)}$ is an orthogonal projection of $\mathbf{x}^{(i)}$ onto subspace S , $(\mathbf{x}^{(i)} - \tilde{\mathbf{x}}^{(i)})$ is orthogonal to every vector in S . The vector $\tilde{\mathbf{x}}^{(i)}$ lies entirely in S . Therefore, $(\mathbf{x}^{(i)} - \tilde{\mathbf{x}}^{(i)})$ is orthogonal to $\tilde{\mathbf{x}}^{(i)}$.

- By the Pythagorean theorem, $\|\mathbf{x}^{(i)} - \tilde{\mathbf{x}}^{(i)}\|^2 + \|\tilde{\mathbf{x}}^{(i)}\|^2 = \|\mathbf{x}^{(i)}\|^2$
- Averaging over data points, we obtain: $\frac{1}{N} \sum_{i=1}^N \|\mathbf{x}^{(i)} - \tilde{\mathbf{x}}^{(i)}\|^2 + \frac{1}{N} \sum_{i=1}^N \|\tilde{\mathbf{x}}^{(i)}\|^2 = \frac{1}{N} \sum_{i=1}^N \|\mathbf{x}^{(i)}\|^2$

12.1 Computing PCA Subspace

Given $\mathbf{x}^{(i)} \in \mathbb{R}^D$, compute $\mathbf{z}^{(i)} \in \mathbb{R}^K$, $K < D$ that preserves as much variance as possible.

1. Center data: $\tilde{\mathbf{x}}^{(i)} = \mathbf{x}^{(i)} - \mu$
2. Compute cov: $\Sigma = \frac{1}{N} \sum_{i=1}^N (\tilde{\mathbf{x}}^{(i)}) (\tilde{\mathbf{x}}^{(i)})^T$
 - a. Cov mat. Captures how much data varies in each direction & how variations in diff. dims relate to each other
3. Find principal components: decompose $\Sigma = Q \Lambda Q^T$, $Q = [\mathbf{u}_1, \mathbf{u}_2 \dots]$ (eigenvectors), $\Lambda = \text{diag}(\lambda_1, \dots)$ (eigenvalues). Σ is PSD $\Rightarrow$ eigenvals real, NN, $Q^{-1} = Q^T$. Then: sort eigenvalues in descending order, select top k corresponding eigenvectors to form $U_k = [\mathbf{u}_1 \dots \mathbf{u}_k]$. U_k span the subspace that maxes proj. var or min. recons err

12.2 Shall Data Be Normalized

(For variance only, always normalize the mean).

DO when: Features have diff. units, features have vastly diff. scales, want equal influence from all features

DON'T when: all features same unit, variance differences are meaningful

12.3 The Good, The Bad

The good:

- Visualize data in 2D/3D space.
- Helps with the curse of dimensionality.
- Eliminate noise or irrelevant info from data.
- Extract new, more informative features.

Limits:

- Information loss
- Sensitive to feature scales.
- Challenging to interpret the meanings of the principal components
- Linear transformation only

13 Ethics

COMPAS is the recidivism prediction tool. Fairness:

- **Individual fairness:** fair if it treats people who are **relevantly similar**, similarly
- **Group fairness:** different demographic groups should receive similar overall outcomes
 - e.g. fair if men/women have similar TPR or FPR
- **Relevant similar:** Probably intent, need, consent, risk. Unlikely: race, ethnicity, arbitrary traits.

Sycophancy and Manipulation

- **Noggle:** Manipulative action is the intentional attempt to get someone's **beliefs, desires, or emotions** to violate **their** norms or ideals, from the perspective of the manipulator. (*Influence* aimed at making people act against what they themselves think is right)
- **Sussur, Roessler, Nissenbaum (SNR):** Causing someone to make non-ideal decisions is sometimes manipulative, but it is not manipulative in every case. Manipulation is imposing a hidden or covert influence on another person's decision-making. *i.e.* influence $\neq$ manipulation unless emotional pressure, deceptive framing, or psych. Tactics are used
- SNR is about the mechanism (is it hidden, does it bypass awareness)? Noggle is about the goal: is the influencer trying to steer someone away from their own values?